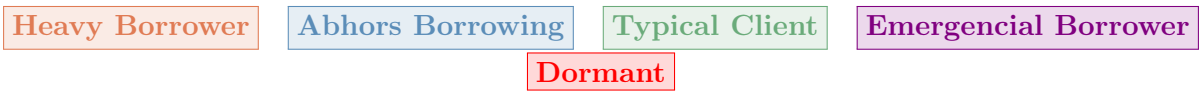

Credit Card Behavioural Clustering:

K-Means vs Fuzzy K-Means on Real Customer Transaction Data

Report

Hard vs Soft Assignment for segmenting clients based on their behaviour,
Validated against expert-crafted rules



Algorithms K-Means • Fuzzy K-Means
Dataset 8950 credit-card holders (Purchases & Cash Advance)

Contents

1	Introduction	2
1.1	Steps of the project	2
2	Dataset	3
2.1	Constructing the "Expert"-defined ground truth	3
3	Clustering Algorithms	5
3.1	K-Means	5
3.2	Fuzzy K-Means	6
3.2.1	From K-Means to FKM	6
4	Results	9
4.1	Crisp K -Means	9
4.2	Fuzzy K -Means	9
4.3	Side-by-Side Visual Comparison	10
4.4	Ambiguous Points	11
4.5	Which one to use?	11
5	Conclusion	13
6	Bloopers	13

1. Introduction

One of the many ways in which banks can generate profits is overdue charges from loans against the credit line of credit cards. This phenomenon is particularly egregious in the US, although it can be relevant in other markets as well, and it makes client segmentation vital: knowing how a client tends to behave helps in evaluating the risk of the loan, which is fundamental in deciding whether a loan request should be accepted or withheld.

This task can be accomplished via clustering, and I picked this as my project precisely because it should be a clear case in which I would expect fuzzy clustering to outperform a crisp one: placing hard bounds between classes is quite difficult, especially as the number of client categories grows. The resulting clusters were then compared to a "ground truth", which is something that would be set by a field expert in a real scenario, and evaluated based on how closely they matched with the "expert"-crafted rules.

1.1. Steps of the project

1. Construct a "ground truth", as an expert would in a real case scenario.
2. Cluster customers via using two features: total purchases and cash-advance.
3. Quantify how well each algorithm's unsupervised partition performs with respect to the rule-based ground truth set at stage 1.

2. Dataset

The dataset is the public *CC GENERAL* credit-card usage dataset, containing 17 behavioural and financial fields per customer over a 6-month observation period. This project uses only two of those fields as clustering features, in order to maintain a visually pleasing result in the plots, but including more dimensions could improve the resulting clusters.

Table 1: Fields used in this analysis

Field	Description
PURCHASES	Total dollar amount of purchases made from the account
CASH_ADVANCE	Total dollar amount of cash borrowed against the credit line
True_Behavior	Derived 5-way label (see Section 2.1)

After dropping rows with missing PURCHASES or CASH_ADVANCE values, **8950 customers** remain. No sampling is applied, as the number of samples is sufficiently low, so the entire cleaned dataset is used for the clustering.

2.1. Constructing the "Expert"-defined ground truth

The dataset did not come with any kind of label attached, so a new set had to be defined in order to evaluate the goodness of one method over the other.

Each customer is assigned to one of five behavioural archetypes using a hand-crafted rule based on the ratio and magnitude of purchases (p) versus cash advances (ca):

- **Heavy Borrower** is a client who relies heavily on cash advances, out of proportion with respect to purchases.
- **Abhors Borrowing** is a client who spends heavily but basically never opens any line of credit.
- **Dormant** is a low activity client on both fronts.
- **Emergencial Borrower** is a client who borrows disproportionately relative to spending, suggesting
- **Typical Client** is a moderate, unremarkable client who makes a balanced use of the service.

$$\text{label}(p, ca) = \begin{cases} \text{Heavy Borrower} & \text{if } ca > 1.5p \text{ and } ca > 4000 \\ \text{Abhors Borrowing} & \text{if } p > 1.5 ca \text{ and } p > 4000 \\ \text{Dormant} & \text{if } p < 1000 \text{ and } ca < 1000 \\ \text{Emergencial Borrower} & \text{if } \frac{ca}{p+1} \geq 0.5 \text{ and } \max(p, ca) > 3000 \\ \text{Typical Client} & \text{otherwise} \end{cases}$$

where $p = \text{PURCHASES}$ and $ca = \text{CASH_ADVANCE}$.

The following are the results of the classification via "expert"-defined rules:

Table 2: Ground-truth class distribution

Behaviour	Count	% of dataset
Dormant	4557	50.9%
Typical Client	3051	34.1%
Heavy Borrower	579	6.5%
Abhors Borrowing	416	4.6%
Emergencial Borrower	347	3.9%
Total	8950	100.0%

This ground truth is itself a simple threshold rule, not an independently verified external label. That being said, it has been crafted so that the non spherical shape, as well as the overlapping between the clusters, should prove hard to handle for our centroid-based clustering algorithms.

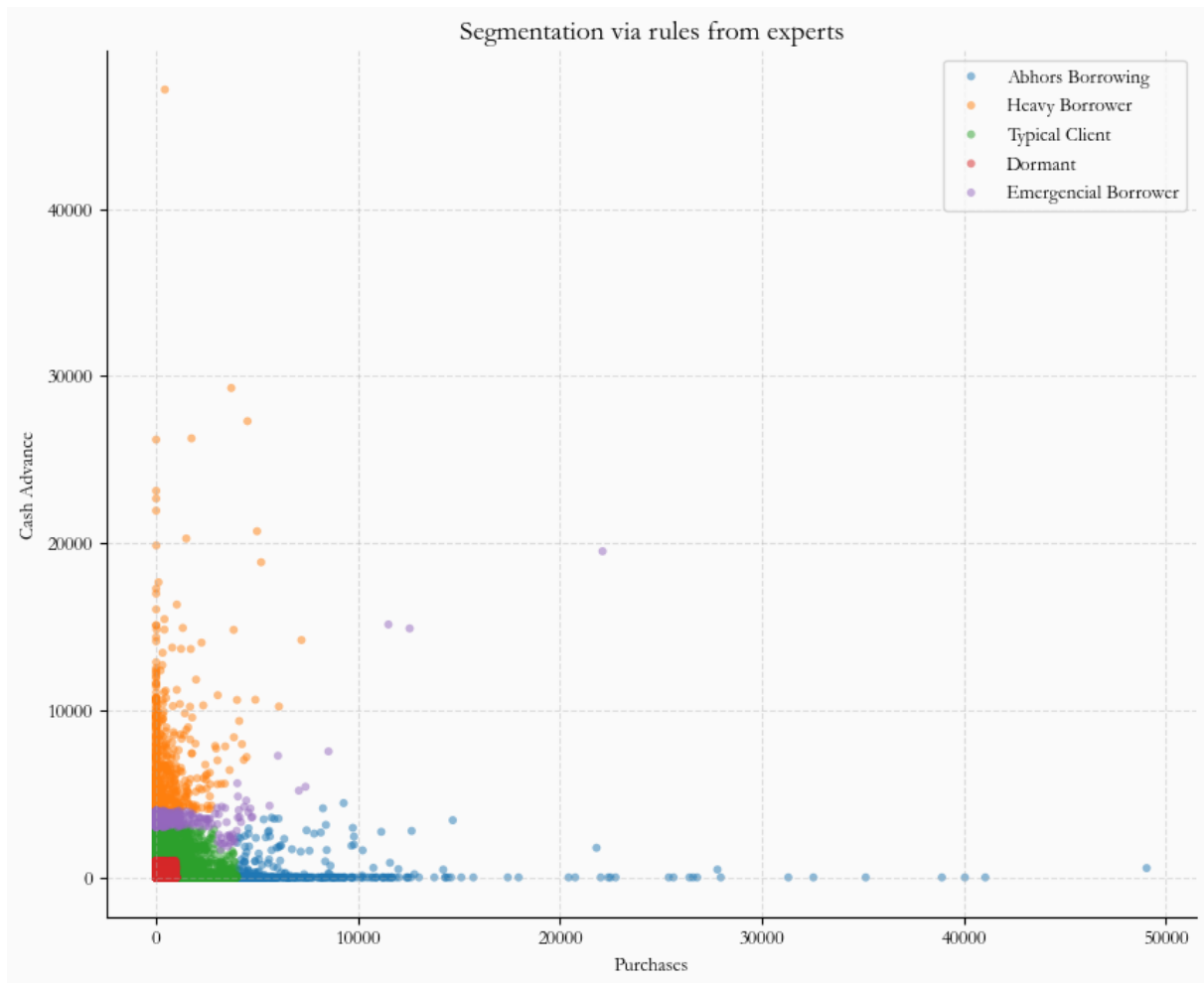


Figure 1: "Expert"-designed rules visualization

3. Clustering Algorithms

Both algorithms were implemented from scratch in NumPy (no `scikit-learn` clustering calls), using pairwise distances from `scipy.spatial.distance.cdist`, a fixed random seed of 42, with a tolerance of 10^{-6} as well as a maximum of 300 iterations as termination criterions.

3.1. K-Means

K-Means minimises the **within-cluster sum of squares** (WCSS):

$$J_{\text{KM}} = \sum_{k=1}^K \sum_{\substack{i=1 \\ z_i=k}}^N \|\mathbf{x}_i - \mathbf{v}_k\|^2$$

via alternating optimisation:

Algorithm 1 K-Means Alternating Optimisation

- 1: Randomly initialise K centroids $\{\mathbf{v}_k\}$
 - 2: **repeat**
 - 3: **Assignment:** $z_i \leftarrow \arg \min_k \|\mathbf{x}_i - \mathbf{v}_k\|^2 \quad \forall i$
 - 4: **Update:** $\mathbf{v}_k \leftarrow \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i \quad \forall k$
 - 5: **until** centroids converge ($\Delta < \varepsilon$) or max tol is reached
-

K-Means: Key Property

Every customer receives a **hard label** $z_i \in \{0, \dots, k-1\}$. A customer who sits between two archetypes, for example moderate purchases combined with a moderate cash advance, is forced into whichever centroid is closest to its position.

```

1 def kmeans(X: np.ndarray, k: int, max_iter: int = 300, tol: float = 1e-6, seed: int = 42):
2     """
3     Classic K-Means via alternating optimisation.
4     Returns: labels, centroids, inertia (WCSS)
5     """
6     np.random.seed(seed)
7     idx = np.random.choice(X.shape[0], size=k, replace=False)
8     centroids = X[idx].copy()
9
10    for _ in range(max_iter):
11        dists = cdist(X, centroids)          # (n, k) pairwise Euclidean distances
12        labels = dists.argmax(axis=1)        # hard assignment
13
14        new_centroids = np.array([
15            X[labels == c].mean(axis=0) if (labels == c).any() else centroids[c]
16            for c in range(k)
17        ])
18        if np.linalg.norm(new_centroids - centroids) < tol:
19            break
20        centroids = new_centroids
21
22    inertia = sum(
23        np.linalg.norm(X[labels == c] - centroids[c]) ** 2
24        for c in range(k) if (labels == c).any()
25    )
26    return labels, centroids, inertia
27

```

Figure 2: kmeans function

3.2. Fuzzy K-Means

Fuzzy *C*-Means relaxes the hard assignment of *K*-Means, replacing it with a continuous **membership** $u_{ik} \in [0, 1]$ of point i in cluster k . It minimises the fuzzy-weighted WCSS:

$$J_{\text{FCM}} = \sum_{k=1}^K \sum_{i=1}^N u_{ik}^m \|\mathbf{x}_i - \mathbf{v}_k\|^2$$

with closed-form update rules:

$$u_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{\|\mathbf{x}_i - \mathbf{v}_k\|}{\|\mathbf{x}_i - \mathbf{v}_j\|} \right)^{\frac{2}{m-1}}} \quad \mathbf{v}_k = \frac{\sum_i u_{ik}^m \mathbf{x}_i}{\sum_i u_{ik}^m}$$

3.2.1. From *K*-Means to FKM

The FKM implementation was done by making three targeted changes to the `kmeans()` function, with the surrounding loop structure, convergence check, and distance computation left identical.

Table 3: Changes from K-Means to FKM

Step	K-Means	Fuzzy C-Means
Initialise	Random centroids	Random membership matrix U (columns sum to 1)
Centroid update	Unweighted mean of hard-assigned members	Fuzzy-weighted mean using u_{ik}^m as weights
Assignment	Hard $\text{argmin} \rightarrow$ one label per point	Soft ratio rule $\rightarrow$ full membership row per point

Partition quality is summarised by the **Fuzzy Partition Coefficient**:

$$\text{FPC} = \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K u_{ik}^2 \in \left[\frac{1}{K}, 1\right]$$

and per-customer ambiguity by **partition entropy**:

$$H_i = - \sum_{k=1}^K u_{ik} \log u_{ik} \in [0, \log K]$$

FKM: Key Property

Every customer receives a partial membership in multiple clusters simultaneously, rather than a forced binary choice, the matrix U tells exactly how confident the algorithm is about each customer .


```

1  def fuzzy_cmeans(X: np.ndarray, k: int, m: float = 2,
2      |             |             | max_iter: int = 300, tol: float = 1e-6, seed: int = 42):
3      |             |             | """
4      |             |             | Fuzzy C-Means, built from kmeans() with minimal changes.
5      |             |             |
6      |             |             | Changes w.r.t kmeans()
7      |             |             | -----
8      |             |             | 1. Initialise U (membership matrix) instead of random centroids.
9      |             |             | 2. Centroid update: fuzzy-weighted mean instead of an unweighted mean.
10     |             |             | 3. Assignment : soft membership rule instead of an hard argmin.
11     |             |             | -----
12     |             |             |
13     |             |             | Everything else is identical to kmeans().
14     |             |             | """
15     |             |             | n = X.shape[0]
16     |             |             | np.random.seed(seed)
17     |             |             |
18     |             |             | # CHANGE 1: initialise membership matrix U instead of centroids
19     |             |             | U = np.random.dirichlet(np.ones(k), size=n).T # shape (k, n)
20     |             |             | centroids = np.zeros((k, X.shape[1])) # computed from U in first iter
21     |             |             |
22     |             |             | for _ in range(max_iter):
23     |             |             |
24     |             |             |     # CHANGE 2: fuzzy-weighted centroid update
25     |             |             |     W = U ** m # (k, n) fuzzified weights
26     |             |             |     centroids = (W @ X) / W.sum(axis=1, keepdims=True) # (k, d)
27     |             |             |
28     |             |             |     # distance matrix -- identical to kmeans
29     |             |             |     dists = cdist(X, centroids) # (n, k)
30     |             |             |
31     |             |             |     # CHANGE 3: soft membership update instead of hard argmin
32     |             |             |     power = 2.0 / (m - 1)
33     |             |             |     U_new = np.zeros((k, n))
34     |             |             |     for i in range(n):
35     |             |             |         d = dists[i] # distances to all k centroids
36     |             |             |         if np.any(d == 0): # point sits on a centroid
37     |             |             |             U_new[:, i] = (d == 0).astype(float)
38     |             |             |         else:
39     |             |             |             ratios = (d[:, None] / d[None, :]) ** power # (k, k)
40     |             |             |             U_new[:, i] = 1.0 / ratios.sum(axis=1)
41     |             |             |
42     |             |             |     # convergence check, same pattern as kmeans but applied to U instead of the centroids
43     |             |             |     if np.linalg.norm(U_new - U) < tol:
44     |             |             |         U = U_new
45     |             |             |         break
46     |             |             |     U = U_new
47     |             |             |
48     |             |             | labels = U.argmax(axis=0)
49     |             |             | fpc = (U ** 2).sum() / n # Fuzzy Partition Coefficient
50     |             |             | return labels, centroids, U, fpc

```

Figure 3: fcm function

4. Results

4.1. Crisp K -Means

Running K -Means with $K = 5$ (matching the number of expert archetypes) gives a WCSS (inertia) of 1.905×10^{10} . The resulting cluster sizes are highly imbalanced, as shown in Table 4, which is expected as a natural consequence of centroid-based clustering on data with one very dense region (the typical clients are expected to be the majority).

Table 4: K -Means cluster sizes ($K = 5$).

Cluster	Points
0	1262
1	176
2	6836
3	652
4	24

Figure 5 places the K -Means assignment side by side with the expert segmentation. Measured against the expert labels, K -Means achieves only **27.4%** agreement, which is unsurprising considering that a single dominant cluster (cluster 2, 6,836 points) absorbs most of the data, whereas the expert rule splits the very same region into *Dormant* and *Typical Client*.

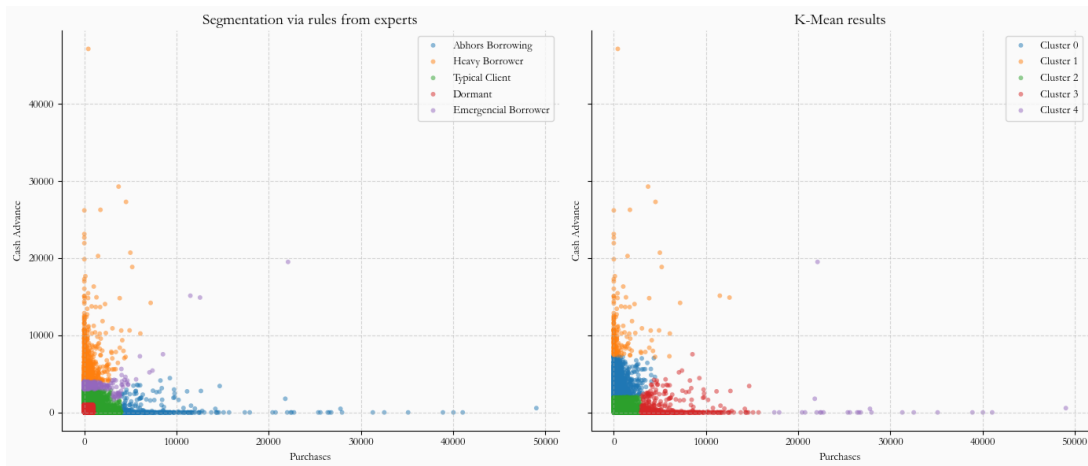


Figure 4: Expert segmentation (left) vs. K -Means hard assignment (right).

4.2. Fuzzy K -Means

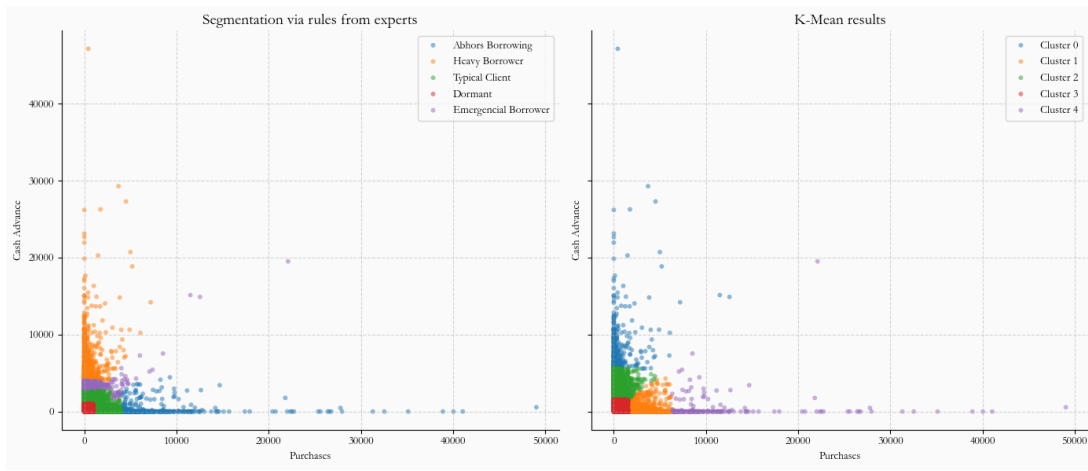
FKM ($K = 5$, $m = 2$) yields a Fuzzy Partition Coefficient (FPC) of **0.7578**, on a scale where 1.0 denotes a perfectly crisp partition and $1/K = 0.2$ denotes a maximally fuzzy one. An FPC of 0.76 indicates a moderately, crisp partition, which in turn means there are indeed a meaningful fraction of points where the mixed membership FKM can shine through.

As printing the full $5 \times 8,950$ membership matrix is impractical, Table 5 shows the difference between the five most *confident* points (lowest membership entropy) against the five most *ambiguous* points (highest membership entropy), where entropy is computed as $H_i = -\sum_k u_{ik} \log u_{ik}$.

Table 5: Most-confident vs. most-ambiguous points from the FKM membership matrix $\mathbf{U}$.

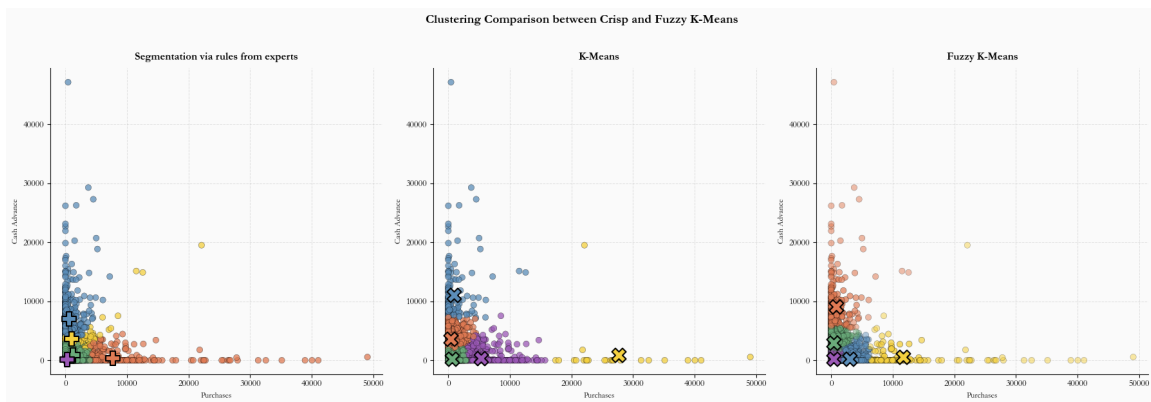
<i>Most confident</i> (5 points)						<i>Most ambiguous</i> (5 points)					
	pt8054	pt8876	pt2386	pt4670	pt342		pt501	pt6258	pt5358	pt550	pt2159
C0	0.0	0.0	0.0	0.0	0.0	C0	0.167	0.157	0.227	0.174	0.264
C1	0.0	0.0	0.0	0.0	0.0	C1	0.195	0.270	0.178	0.196	0.179
C2	0.0	0.0	0.0	0.0	0.0	C2	0.171	0.223	0.177	0.177	0.202
C3	1.0	1.0	1.0	1.0	1.0	C3	0.172	0.151	0.157	0.178	0.179
C4	0.0	0.0	0.0	0.0	0.0	C4	0.295	0.200	0.260	0.275	0.174

The most confident points sit exactly on a centroid, receiving membership 1.0 in a single cluster. The most ambiguous points, by contrast, have memberships close to $1/K = 0.2$ spread across *all* five clusters: these are customers roughly equidistant from every archetype centroid.

**Figure 5:** Expert segmentation (left) vs. FKM assignment (right).

4.3. Side-by-Side Visual Comparison

Figure 6 places the ground truth alongside both algorithms' output on the same axes, using the same colour palette.

**Figure 6:** Ground truth (left) vs K-Means hard assignment (centre) vs Fuzzy C-Means soft assignment (right). X markers denote centroids, while point opacity in the FKM panel encodes the maximum membership value.

4.4. Ambiguous Points

Cluster indices produced by K -Means and FKM are arbitrary (cluster “2” has no inherent relationship to expert label “Typical Client”), which is also why the cluster color held no semantic meaning in the previous visualizations. While this might not have proven particularly problematic earlier, it could be so in the next visualizations so it needs some fixing.

In order to re-align algorithm clusters to expert archetypes, we use the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`) by Khun, to find the one-to-one cluster-to-archetype mapping that maximises point overlap between each cluster and the one defined by the "expert"-crafted rules.

Using this mapping, Figure 7 shows the 30 points with the highest FKM membership entropy (diamond markers) and their assignment by the algorithms, plotted over the expert segmentation in the background. The mean entropy of these 30 points is **1.565**, close to the theoretical maximum of $\ln(5) = 1.609$, confirming these are close to maximally ambiguous under FCM (near-uniform membership across all five clusters).

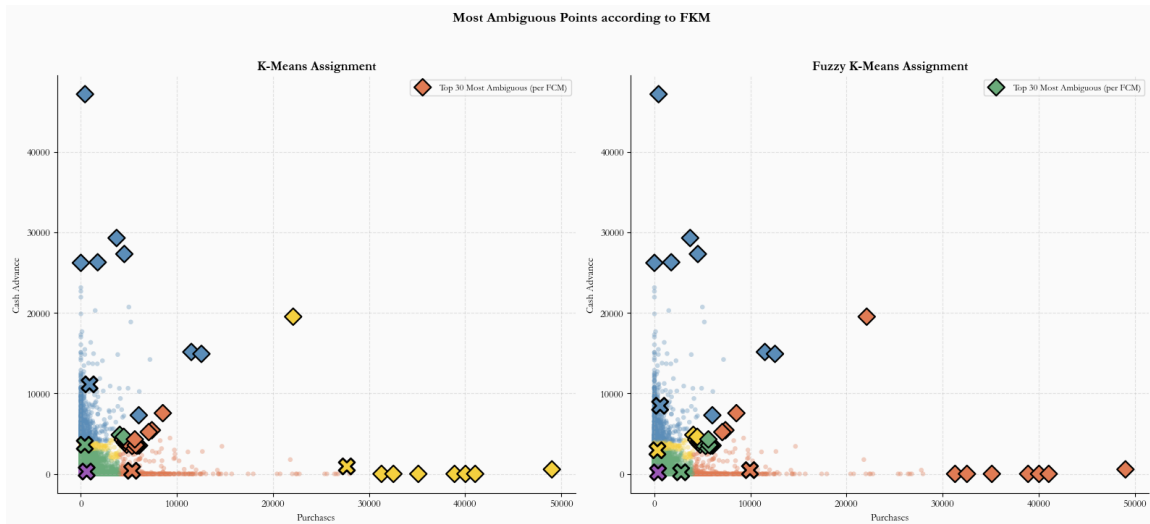


Figure 7: The 30 most ambiguous points per FKM (diamonds) and how they were classified

Not all of the 30 most-ambiguous points show an agreement between K -Means and FKM, and it is particularly evident when the points seat near the middle point between a couple of centroids. It is also of note that K

4.5. Which one to use?

Now that we have seen the two algorithms and the results of their application, it has come the time to finally answer to the main topic of the project: is FKM preferable to K -Means?

To answer that question we will refer to 2 similarity measures between clusters:

- `adjusted_rand_score`, which considers all pairs of samples and counts the ones assigned in the same or in different clusters w.r.t the referenced clustering.
- `normalized_mutual_info_score`, which evaluates the correlation between the two clusters.

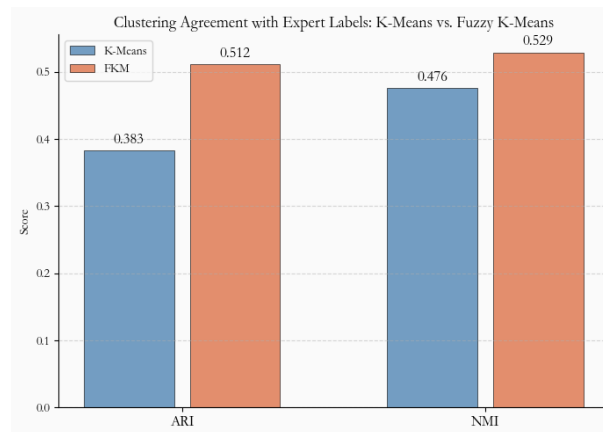


Figure 8: Similarity Measures of the algorithms w.r.t the rule-based clustering

Figure 8 shows the similarity measures of the resulting clusterings with respect to the one defined via the "expert" rules. These results confirm what we could already glimpse from the graphical reappresentations, that is Fuzzy K Means provides a more similar clustering than Crisp K-Means.

5. Conclusion

This report applied the K-Means and Fuzzy K-Means clustering techniques to a real financial dataset of 8950 credit-card holders, validated against an interpretable rule-based ground truth.

Ultimately, Fuzzy K-Means proved to be superior than K-Means, being closer to the original "expert"-defined rules that captures a semantic meaning in the data, even though the Fuzzy Partition Coefficient was relatively high.

6. Bloopers

This was not the project I had chosen at the beginning: I had originally planned to showcase the use of Fuzzy Cluster with respect to [ABC's famous 1 million dataset](#), but then I met this:

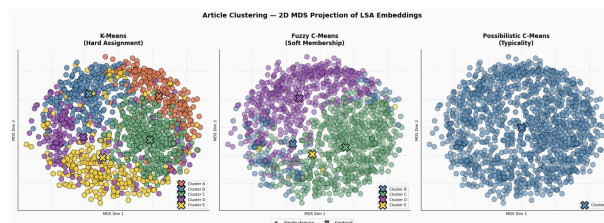


Figure 9: opsie

This is the natural consequence of plotting multi dimensional data after reducing it to 2 dimensions via PCA, and so I decided to change the topic of the project.